



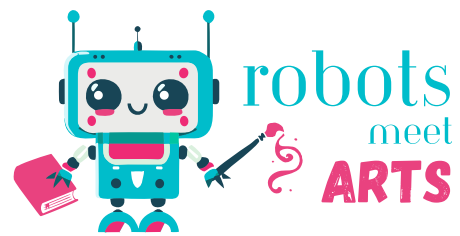
Co-funded by
the European Union

Project number: 2023-1-
FR01-KA220-SCH-
000151881

BORN YESTERDAY

**Basic of Robotics through an unplugged-
to-plugged Bio-Inspired Game**





Subject: Robotics, Biology, Cognition

Content: Machine learning, bio-inspired algorithms, reinforcement learning, adaptability, human cognition

Computational practices:

Decomposition, pattern recognition, abstraction, algorithms, problem solving; block programming, tangible programming, introduction to AI, simulation

Unplugged/plugged:
unplugged to plugged



Preparation time:

60 - 90 min



Age:

8-12 years



Duration: 3 hours



Location:

indoor



Group distribution:

full classroom/in group



Difficulty



This sequence allows pupils to discover the fundamental principles of machine learning by experiencing for themselves an algorithm that learns through trial and error. By combining an immersive unplugged phase, observation of an AI that learns, and a robotics programming phase, pupils understand how machines can "learn" to solve problems without being explicitly programmed for each situation.

The following competencies are developed:

- **"Practising scientific and technological approaches"** – Pupils formulate hypotheses about the "good routes", verify them through experimentation, and discover the principle of reinforcement learning
- **"Designing and producing a technical object"** – Programming robots to complete a course using knowledge acquired during previous phases
- **"Appropriating tools and methods"** – Creating "mental maps" of the course, observing AI simulation tools, collaborative work
- **"Practising languages"** – Expressing exploration strategies, verbalising discovered algorithms, comparing human/machine



Cross-Curricular Links

- **Mathematics:** Work on locating positions on a grid, coordinates, conditional logic (if... then...). Intuitive notion of probability when making exploration choices.
- **Geography:** The navigation context allows addressing themes of spatial orientation and mapping.
- **Citizenship Education:** Discussion on AI ethics: how do autonomous cars learn? Can we trust a machine that learns?
- **Mother tongue:** Oral expression during reflection phases, argumentation to justify strategic choices.

Helpful links:

- BioLearningGame – AI simulation tool: <https://steamcity.github.io/BioLearningGame/>
- Machine Learning for Kids: <https://machinelearningforkids.co.uk>
- Cubetto – Documentation: <https://www.primotoys.com>
- Sphero Indi – Teaching guide: <https://sphero.com/pages/indi>
- MakeCode micro:bit: <https://makecode.microbit.org>
- KittenBot TabbyBot Programming Robotics Kit for micro:bit: <https://www.kittenbot.cc/products/kittenbot-tabbybot-programming-robotics-kit>
- Maqueen Robot for micro:bit: <https://www.kubii.com/fr/robots-extensions/2675-micro-bit-robot-maqueen-kubii-6959420915002.html>
- CS Unplugged – Unplugged activities: <https://csunplugged.org>

Material Needed

- **6x6 Grid** : Several options depending on your resources and space:
 - Giant floor version: Draw a grid with coloured tape (squares of approximately 30–40 cm). You can even use one of the robots as a "counter" that pupils move!
 - A3/A4 paper version: Print the grids provided in the appendix or create your own. Use repurposed board game pieces (chess, draughts, Monopoly...) to simulate the autonomous agent's movements.
- **Memorisation Sheets** : One per team to record discoveries (safe squares, obstacles) – template provided in appendix
- **Rewards** : Sweets, pom-poms, tokens or points... to simulate positive and negative reinforcement
- **Computer/Tablet** : For programming the robots and accessing the BioLearningGame tool developed as part of the Erasmus+ SteamCity project: <https://steamcity.github.io/BioLearningGame/> – "AI" tab
- **Projector** : For collective demonstration of AI learning
- **Robots for the Programming Section** : If possible, pupils rotate through 3 robotics workshops, each offering a different programming logic with specific challenges. We recommend the following tools for a truly diverse programming experience:

Workshop	Robot	Programming Logic	Specific Challenge
Workshop A	Cubetto	Tangible programming: physical blocks placed on a board	Optimise with a limited number of blocks (12 max)
Workshop B	Sphero Indi	Visual colour programming: coloured tiles that the robot reads	Understanding conditional behaviours (IF colour THEN action)
Workshop C	micro:bit Robot (Maqueen, Move Mini...)	On-screen block programming (MakeCode) – Example programme in appendix	Precisely calibrate movements, use sensors

Adaptations for Material Constraints

- **With only 1 robot**: The activity works perfectly well with just one type of robot. Choose the one that matches your budget and objectives. Sphero Indi is very accessible for beginners; the micro:bit robot offers more possibilities but requires a bit more familiarisation. Cubetto is interesting because it offers a puzzle-like challenge for pupils, but it is more expensive and has limited reusability as it is designed for very young children.
- **With 2 robots**: You can compare two different programming logics, which is already very rich pedagogically.
- **No robot?** Contact science outreach associations in your area (science centres, fab labs, maker spaces, libraries, museums...). Many offer robotics equipment loans or can come to your classroom with their equipment.

Introduction

This sequence is an **ideal introductory activity to the "Robots Meet Arts" universe** (Robots and Humanities Encounter). It allows for demystifying robotics programming in a fun way, showing that programming a robot is not reserved for experts: it is accessible, creative, and deeply connected to what we are as human beings.

The starting point: Imagine a robot born yesterday that must learn to navigate an unknown environment to reach a goal. Nobody has given it a map: it must discover for itself which paths are safe and which are blocked by obstacles. How can it learn without getting permanently lost?

In this sequence, pupils will first play the role of this explorer robot, then observe how a real AI solves the same problem, and finally programme their own robots to complete a course without errors. They will discover that by exploring methodically, by memorising what works and what doesn't, they build an effective "mental map" of the territory – exactly as machine learning algorithms do when they build their model.

The originality of this activity lies in its transdisciplinary approach:

Discipline	Digital Sciences	Life Sciences	Humanities
Key Concepts	Robotics programming, algorithms, AI	Bio-inspired models, learning models of living beings	Psychology of learning, human cognition
Questions Explored	How do we programme a robot?	How do living beings learn?	How do we learn as human beings?

Bio-Inspired Learning – When Machines Imitate Life: Learning is one of the fundamental processes shared by humans and machines, even if the mechanisms can be very different. When a child touches a hot plate and learns to avoid it, when they discover through successive attempts the best route home from school – they are practising reinforcement learning. This is exactly what the most powerful AI algorithms do today.

The aim of this activity is to provide pupils with an intuitive understanding of bio-inspired learning models and how machines, like living organisms, use trial and error to adapt and find solutions.

A Fun Entry Point into Robotics: Robotics is not just about code and circuits. It is also an invitation to question our humanity: what is learning? What differentiates human intelligence from artificial intelligence? Can robots truly "understand" or do they just calculate? These philosophical questions emerge naturally from the activity.

Introduction

This activity was designed as an accessible gateway to robotics programming. By starting with an "unplugged" phase where pupils are themselves the robot, we remove apprehensions: programming is simply giving clear instructions, like those they followed during the game. The transition to real robots then becomes natural and non-intimidating.

Three robots are proposed (Cubetto, Sphero Indi, micro:bit), each with a different programming logic. Ideally, all three should be discovered through rotating workshops, but the activity works very well with a single robot depending on your resources. The essential thing is to experience giving instructions to a machine and seeing your creation come to life.

Preparation and teacher notes

- Set up the grids (2–3 depending on class size)
 - on the floor or on tables
- Prepare rewards for reinforcement (sweets, tokens, pom-poms... depending on your approach)
- Print and laminate the memorisation sheets to distribute to pupils – laminating allows them to be used with dry-wipe markers for reuse or correcting noted information
- Have the plan of each grid to hand (to give feedback)
- Provide one counter per group (figurine, switched-off robot, or pupil-counter)

Warm-up: Discovering Human Learning (Unplugged Game)

Objective: Understand trial-and-error learning, and build your own reinforcement learning model.

Form 2 or 3 groups depending on class size (ideally 10–12 pupils per grid). Each group will work around its own grid. If possible, create the grids on the floor, as they will be useful during subsequent phases. Create or select a grid that matches your objectives: more or fewer obstacles, inaccessible zones, available time... Several examples are available for printing in the appendix.

GRID EXAMPLE #1



Game Organisation: Gather each group around its grid. Announce the context without revealing the objective: "You are a robot that must navigate this grid. Your mission: find a specific point. The problem? You don't know where it is! You will have to learn for yourselves by exploring." Give no information about the objective or the layout of obstacles. Pupils must discover for themselves, exactly like a machine that learns without prior knowledge.

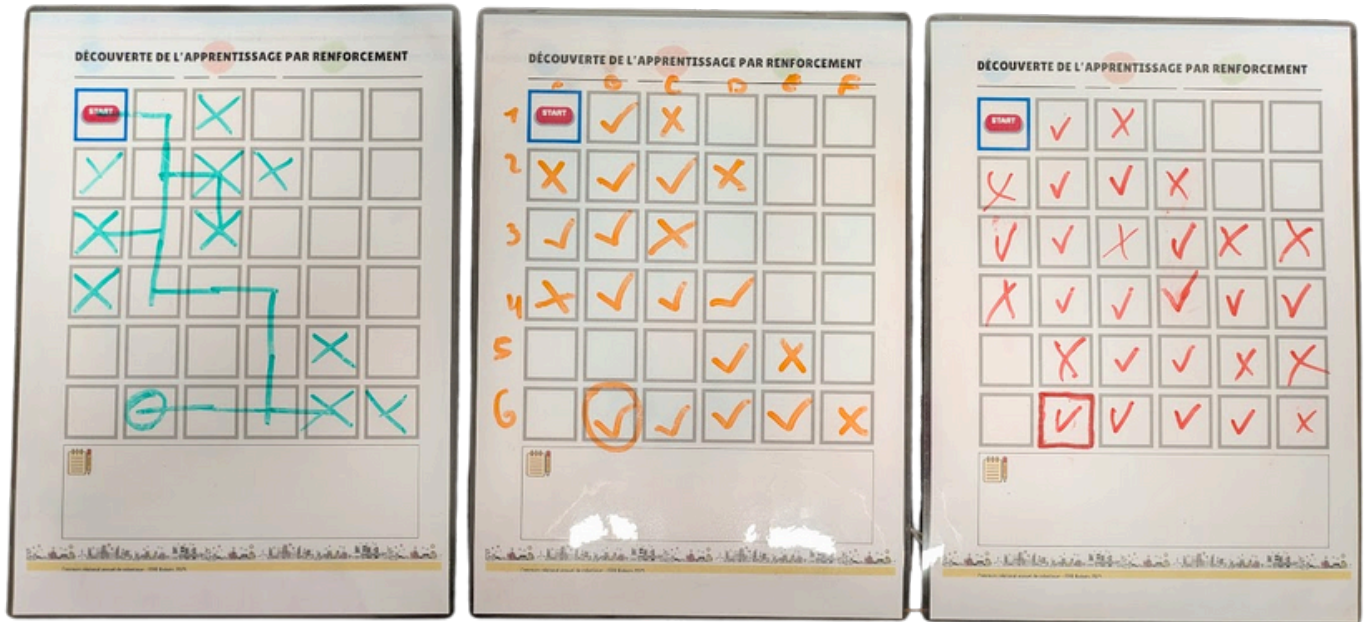
Once groups have found the path, count the tokens! Introduce the scientific vocabulary.

MEMORISATION SHEET

A 6x6 grid of squares. The top-left square (row 1, column 1) contains a red button with the word "START" in white capital letters. The button has a blue border and a slight 3D effect. The remaining 35 squares in the grid are empty.

NOTES

Each pupil keeps their memorisation sheet. Announce: "You are now going to learn to programme a robot. Later, you will have to programme this robot to follow the path you discovered. Your memorisation sheet will be your only guide!"



Example of three different memorisation sheet filled in for the same grid

Crucial Teaching Point – Give NO instructions on how to fill in the sheet. No model, no example, no suggestion. The aim is to observe how each pupil spontaneously builds their own representation system: some will draw arrows to indicate the path, others will put crosses or X on obstacles, others will use colours (green = safe, red = danger)... This diversity IS the learning. Even if all pupils experience the same collective exploration, each builds their own "model" with their own logic. This also applies to movements on the grid. Give no indication of the number of obstacles or clues about the objective's position. For example, some groups of pupils presuppose that the objective is in a corner of the map, on a specific square... it is interesting to observe these biases and discuss them once the activity is finished.

Preparation and teacher notes

- Check that robots are charged and functional
- Prepare the workspace (clear floor area)
- Print the mission sheets corresponding to the robot(s) used
- Have read the teacher tutorial for the robot(s)
- If Phase 1 warm-up was done on paper grids, create a larger floor version of the grid (tape, 30–40 cm/square)

Build-up: Robotics Mission

Objective: Master the basics of robotics programming through progressive missions.

This stage is a technical discovery phase deliberately disconnected from the main challenge. Pupils learn to handle the robot without pressure, through progressive missions that have nothing to do with the Warm-up grid. This separation is intentional:

- Avoid cognitive overload: learning a new tool AND solving a complex problem at the same time would be too difficult
- Build confidence: pupils arrive at the final phase already mastering their robot
- Allow for error: during this stage, making mistakes has no consequence on the final challenge

Whatever robot is used, the progressive missions allow discovering:

1. The basic principle: how to give an instruction to the robot
2. Movements: moving forward, turning
3. Sequencing: combining several instructions in sequence
4. Constraints: each robot has its limits and its own logic

Pupils also discover that a robot doesn't "understand" anything – it executes exactly what it's told. If the robot does something unexpected, it's because the instruction wasn't what you thought you were giving. This realisation is fundamental for what follows.

If you can access all three proposed robots, organise rotating workshops. Each group successively discovers different ways of "speaking" to a robot.

If you only have access to one robot, you can organise other workshops in parallel, during time dedicated to digital or other creative activities, to avoid overcrowding pupils around a single technical tool.

The order of workshops between different robots doesn't matter, because what counts is experiencing different programming logics with several learning objectives:

- **Deconstruct the idea that there's only one way to programme**
- **Understand that "programming" = giving instructions, whatever the form (blocks, tiles, code)**
- **Enrich the discussion: "Which robot did you prefer? Why?" reveals different learning profiles**

There is no hierarchy! Cubetto is not "easier" than micro:bit, they are different. Each robot has its own logic, and mastering one doesn't necessarily help with another. This diversity is what makes the activity rich.

Robot	Programming	Constraints
Cubetto	Physical wooden blocks placed on a board – tangible programming	Limited number of blocks (12 max) → forces planning and optimisation
Sphero Indi	Coloured tiles placed on the floor – visual colour programming	Depends on available tiles, predefined behaviours by colour
micro:bit Robot	On-screen blocks (MakeCode) then download – block programming	Calibrating durations and distances

Let pupils explore using the mission sheets. Some missions require using the 6×6 grid discovered during the warm-up to prepare for the final mission, i.e., creating a programme that allows the robot to autonomously complete the course discovered in Phase 1. If you used paper grids during the Warm-up, you will need to create a 6×6 floor grid using coloured tape, with squares of 30–40 cm adapted to the robot's size. If you had already used a floor grid for Phase 1, don't remove it before the activity is completely finished.

Cubetto Exception: Cubetto comes with its own 6×6 mat perfectly calibrated for its movements. Use this mat directly because Cubetto cannot be calibrated in terms of movement distance!

The missions sheets are available in appendix. While pupils complete the missions, circulate between groups and value mistakes: "Your robot turned the wrong way? Great, you've just learnt something!". Observe strategies: who plans? who tests directly? who collaborates?

MISSIONS: CUBETTO



MISSION 1
FIRST CONTACT
 

Move Cubetto forward 3 spaces in a straight line.

MISSION 2
THE ROTATION
 

Rotate Cubetto to the right, then move forward 2 spaces.

MISSION 3
REVERSE GEAR MISSION
 

Return Cubetto to its starting point.

MISSION 4
L-SHAPED ROUTE
 

Program a path from point A to point B that forms an L.

MISSION 5
MYSTERY PATTERN
 

Repeat the "forward-turn right" movement twice.

MISSION 6
THE PERFECT SQUARE
 

Program Cubetto to draw a 3-3 square.

MISSION 7
THE STAIRCASE
 

Program Cubetto to go down the stairs for as long as possible.

MISSION 8
SECRET CODE
 

Program Cubetto to draw a T.

MISSION 9
THE GRID
 

Create a program that traverses as many squares of the grid as possible.

MISSIONS: SPHERO INDI



MISSION 1
STRAIGHT LINE
 

Move Indi forward 3 spaces in a straight line.

MISSION 2
ROUND TRIP
 

Move Indi forward and then make him return to his starting point.

MISSION 3
TURN RIGHT
 

Turn Indi 90° to the right, and then move forward.

MISSION 4
STOP!
 

Move Indi forward and then stop him precisely.

MISSION 5
SPEEDY
 

Create a course with changes in speed.

MISSION 6
FLUID SLOMO
 

Slo-mo through a course with 3 obstacles, the start and finish of which are located on the same road.

MISSION 7
THE S-SHAPED ROUTE
 

Create an S-shaped path in a minimum area of one square meter.

MISSION 8
THE STAR
 

On the grid, start from the center and visit the 4 edges of the grid.

MISSION 9
THE GRID
 

Create a program that traverses as many squares of the grid as possible.

MISSIONS: MICRO-BIT



MISSION 1
HELLO ROBOT
 

Move the robot forward in a straight line of at least 2 meters.

MISSION 2
ROBOT BALLET
 

Make the robot dance in place (in a graceful way!).

MISSION 3
CALIBRATION
 

Find out how to move your robot just one space forward on the grid.

MISSION 4
THE TURNS
 

Go through the grid, move forward, turn right, move forward, turn left.

MISSION 5
THE L
 

On the grid, 3 squares forward, right, 2 squares forward.

MISSION 6
THE SQUARE
 

On a grid, create a 3-3 square using a loop.

MISSION 7
THE LEDS
 

Move the robot forward 6 times with only 2 lines of code.

MISSION 8
THE FUNCTIONS
 

Complete the previous mission using the functions.

MISSION 9
RESPONDING
 

Add messages using LEDs and sounds while on the go.

Preparation and teacher notes

We recommend completing this phase in parallel with Phase 2.

It is best to offer this stage as the final mission continuing from the mission sheets of the previous stage.

Thus, each group follows this cycle:

- Robot Workshop A: Missions → Rehearsal Challenge → Rotation
- Robot Workshop B: Missions → Rehearsal Challenge → Rotation
- Robot Workshop C: Missions → Rehearsal Challenge → Rotation

Rehearsal: Transferring the Bio-Inspired Learning Model to Robots

Objective: Use your memorisation sheet to programme the robot to follow the path discovered during the Warm-up.

This is the moment of testing the pupils' models!

Pupils must programme their robot to follow the path they discovered during the Warm-up. The golden rule: They have **ONLY** their memorisation sheet. They cannot see the complete grid with its obstacles before programming!

Pupils study their memorisation sheet and plan their programme:

- What path had they found?
- What instructions should they give the robot?
- In what order?

This is where the quality of the memorisation sheet is revealed! Some pupils will have clearly noted the path, others will have to "decode" their own notes.

Pupils programme their robot based solely on their sheet. They will not be allowed to touch their robot once their programme is launched, so it must traverse the grid autonomously.

To validate their programme, the group places their robot at the start of the grid, launches the programme, and the class observes whether their agent follows the correct path and reaches the objective. If the robot fails, let the group correct their programme and test again.

By programming the same path with different robots, pupils realise that the same logic (their sheet) is expressed differently depending on the tool, and some robots are better suited to certain courses. Their memorisation sheet must be clear enough to be "translated" into any language.

Preparation and teacher notes

- Gather all pupils' memorisation sheets
- Prepare a display space (board, wall, floor) to show the different sheets
- Keep in mind observations made during the Warm-up (diversity of notation systems)
- Prepare computer + projector for BioLearningGame
- Test access to <https://steamcity.github.io/BioLearningGame/> before the session

Clean-up: Reflection and Discussion

Objective: Analyse the learning, compare memorisation strategies, observe how an AI learns, make the human/machine connection.

This is the moment to share and compare the different models created during the activity and to understand our learning modes by comparing them to those of a machine. Display the sheets (or pass them around) and lead a discussion:

- "What notation systems did you invent?"
- "Which ones worked well for programming the robot? Why?"
- "Were there difficulties rereading your own notes?"
- "If you had to do the activity again, what would you change on your sheet?"
-

Now that pupils have experienced learning a path and memorising it, show them how artificial intelligence solves the same problem.

Project

BioLearningGame
(<https://steamcity.github.io/BioLearningGame/>),

specifically the tab called "**AI Part**". Pupils observe that at the beginning, the AI moves randomly, like them at the start of the first phase. It makes many more mistakes than humans: it goes off the grid, goes backwards, goes around in circles. It tests the grid like a maze – for it, a square can be an obstacle if entered from the left, but not if entered from above! Gradually, it avoids obstacles it has encountered, and eventually finds the optimal path.

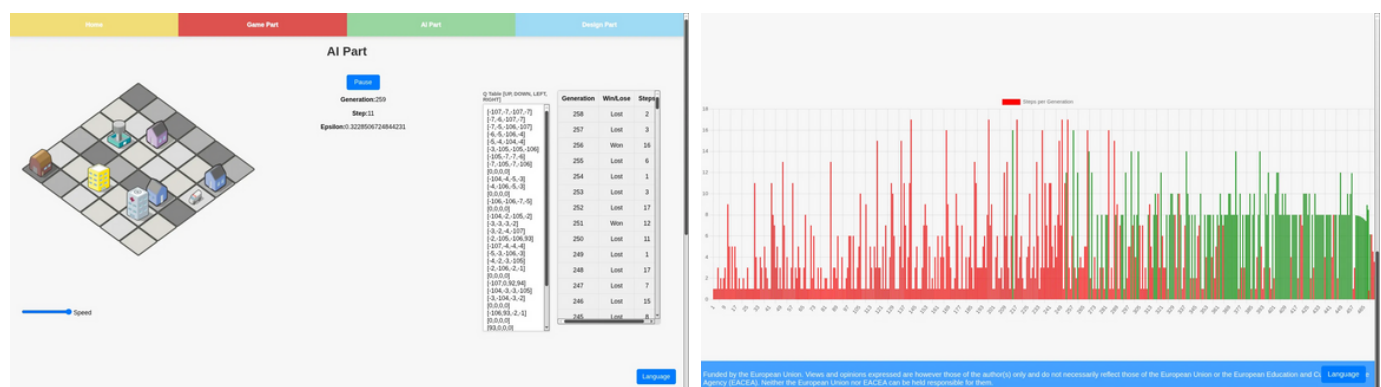
The numbers are surprising: the AI can take up to 700 attempts to find the optimal path, but it makes these 700 attempts in less than a minute. Pupils, meanwhile, found it in a few attempts... but it took them much longer. The AI compensates for its ignorance of the environment with its speed.

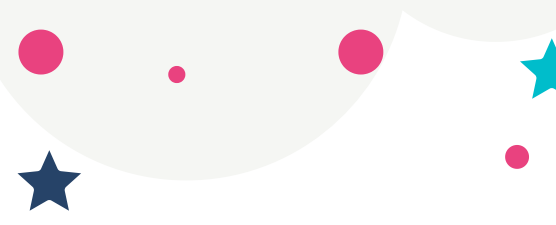
Also observe what the AI is trying to optimise: it wants the best possible score, i.e., reaching the objective with the minimum number of steps. It's not just looking for "a path that works", it's looking for "the best path". That's why it continues exploring even after finding a solution.

During observation, ask questions: How does the AI move at the beginning? (Randomly, it even goes off the grid!) Why does it make so many mistakes? (It has no intuition, it tests EVERYTHING) How many attempts does it need? (Hundreds!) Why does it continue after finding a path? (It's looking for the optimal path, the shortest one). Then lead a comparative discussion. Pupils found the path in a few attempts, the AI in 700. Pupils took several minutes, the AI less than a minute. Pupils think before acting, the AI tries everything without thinking. Pupils stay on the grid (logical!), the AI goes off the grid and tests everything. Pupils use their intuition, the AI only has calculations. Pupils look for "a path that works", the AI looks for "the optimal path". Pupils can explain their strategy, the AI cannot. Pupils adapt easily to a new problem, the AI must relearn everything from scratch.

Pose discussion questions: Who learnt faster, you or the AI? (It depends how you count! Fewer attempts for you, less time for the AI) What are the AI's advantages? (Doesn't get tired, very fast, finds the optimal solution, doesn't get discouraged) What are YOUR advantages? (Intuition, few errors, ability to explain, to adapt, to understand) In which cases would the AI be better? And you?

Conclude on the richness of what they have experienced: "Today, you did much more than programme a robot. You discovered how YOU learn, you invented your own notation system, and you understood how machines are inspired by living beings' learning.". Open up to broader questions: Where do we find this type of learning in everyday life? (GPS, video games, robot vacuum cleaners...) Can robots truly "understand" or do they just calculate? For the written record, create together a poster or summary document with the different notation systems invented by the class, key words (reinforcement learning, mental model, trial and error), a comparative human/machine diagram, and pupils' reflections.



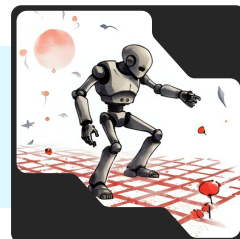


Appendixes - Resources for the Teachers and Students

1. **Memorisation Sheet:** *1 sheet per student or per sub-group*
2. **Grids:** *10 version of the grid with different level of difficulties and labyrinths. You can create yours using a blank memorisation sheet.*
3. **Missions:** *3 missions sheets for each robot used in the activity.*



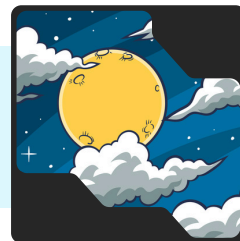
MEMORISATION SHEET

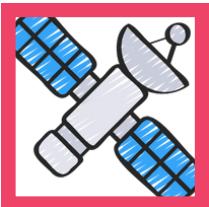


START					

NOTES

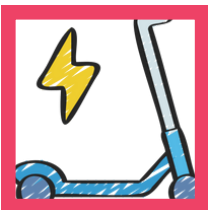
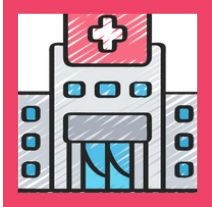
GRID EXAMPLE #1



	A	B	C	D	A	F
1						
2						
3						
4						
5						
6						

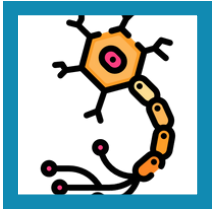
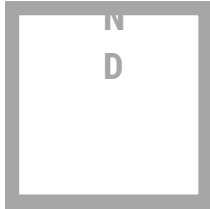
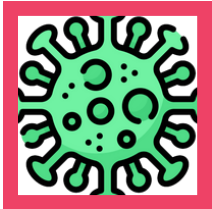
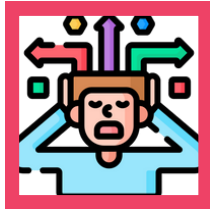
GRID EXAMPLE #2



	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

GRID EXAMPLE #3



	A	B	C	D	A	F
1						
2						
3						
4						
5						
6						

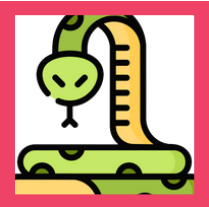
GRID EXAMPLE #4



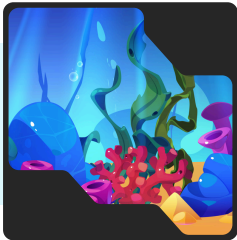
	A	B	C	D	A	F
1						
2						
3						
4						
5						
6						

GRID EXAMPLE #5



	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

GRiD EXAMPLE #6



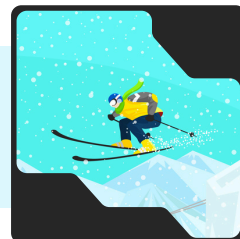
	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

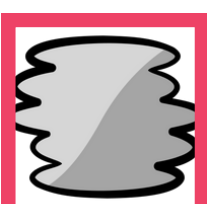
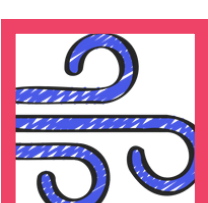
GRID EXAMPLE #7



	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

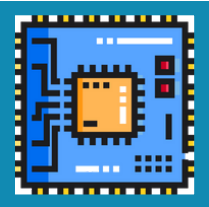
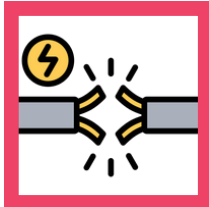
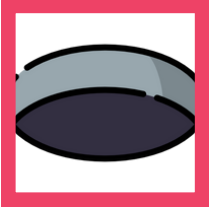
GRID EXAMPLE #8



	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

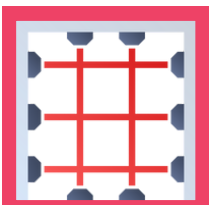
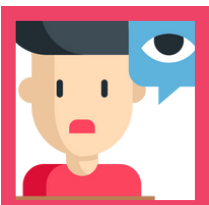
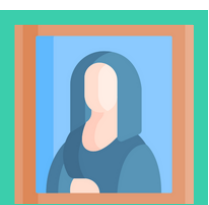
GRiD EXAMPLE #9



	A	B	C	D	A	F
1					N D	
2						
3						
4						
5						
6						

GRID EXAMPLE #10



	A	B	C	D	A	F
1						
2						
3						
4						
5						
6						

MISSIONS: CUBETTO



MISSION 1 FIRST CONTACT

Move Cubetto forward 3 spaces in a straight line.



MISSION 2 THE ROTATION

Rotate Cubetto to the right, then move forward 2 spaces.



MISSION 3 REVERSE GEAR MISSION

Return Cubetto to its starting point.



MISSION 4 L-SHAPED ROUTE

Program a path from point A to point B that forms an L.



MISSION 5 MYSTERY PATTERN

Repeat the "forward-turn right" movement twice.



MISSION 6 THE PERFECT SQUARE

Program Cubetto to draw a 3x3 square.



MISSION 7 THE STAIRCASE

Program Cubetto to go down the stairs for as long as possible



MISSION 8 SECRET CODE

Program Cubetto to draw a T.



MISSION 9 THE GRID

Create a program that traverses as many squares of the grid as possible.



MISSIONS: SPHERO iNDi



MISSION 1 STRAIGHT LINE



Move Indi forward 3 spaces in a straight line.

MISSION 2 ROUND TRIP



Move Indi forward and then make him return to his starting point.

MISSION 3 TURN RIGHT



Turn Indi 90° to the right and then move forward.

MISSION 4 STOP !



Move Indi forward and then stop him precisely.

MISSION 5 SPEEDY



Create a course with changes in speed.

MISSION 6 FLUID SLALOM



Slalom through a course with 3 obstacles, the start and finish of which are located on the same road.

MISSION 7 THE S-SHAPED ROUTE



Create an S-shaped path in a minimum area of one square meter.

MISSION 8 THE STAR



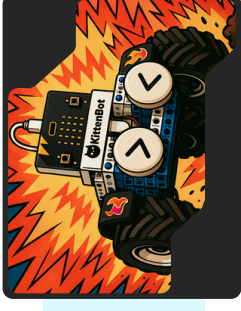
On the grid: start from the center and visit the 4 edges of the grid.

MISSION 9 THE GRID



Create a program that traverses as many squares of the grid as possible.

MISSIONS: MICRO:BIT



MISSION 1 HELLO ROBOT

Move the robot forward in a straight line of at least 2 meters.



MISSION 2 ROBOT BALLET

Make the robot dance in place (in a graceful way).



MISSION 3 CALIBRATION

Find out how to move your robot just one space forward on the grid.



MISSION 4 THE TURNS

Go through the grid: move forward, turn right, move forward, turn left.



MISSION 5 THE L

On the grid: 3 squares forward, right, 2 squares forward.



MISSION 6 THE SQUARE

On a grid: create a 3x3 square using a loop.



MISSION 7 THE LOOPS

Move the robot forward 6 times with only 2 lines of code.



MISSION 8 THE FUNCTIONS

Complete the previous mission using the functions.



MISSION 9 DISPLAYING

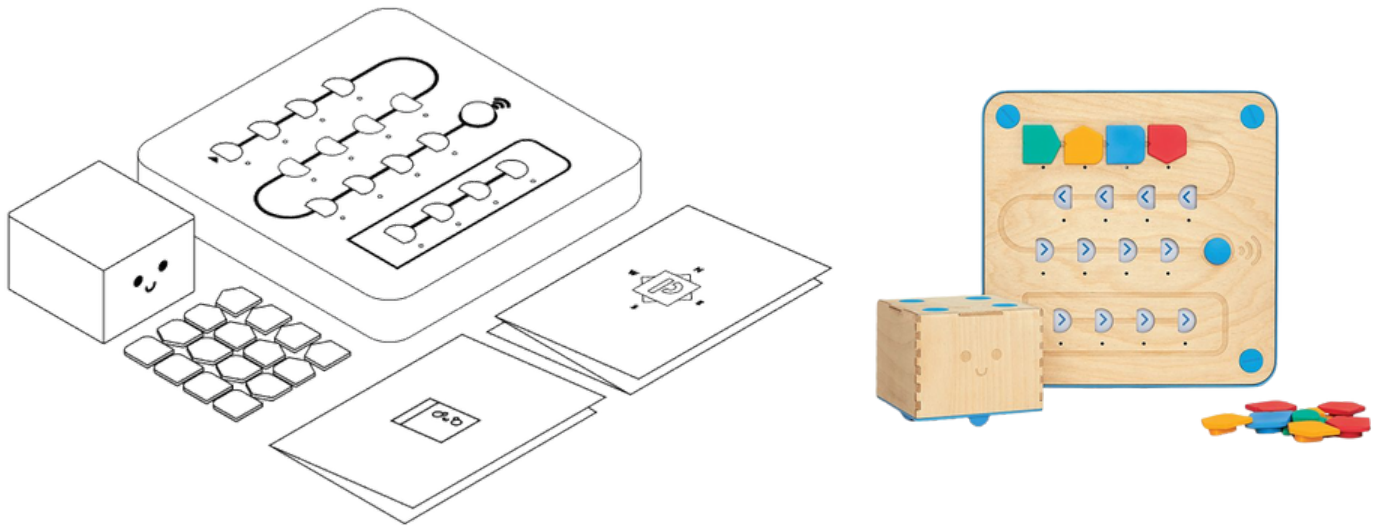
Add messages using LEDs and sounds while on the go.



Appendixes - Tutorials of Robots Used in the Activity

Cubetto

Source: <https://primotoys.com/cubetto-user-manual/>



Cubetto is a tangible programming interface designed to introduce children aged 3 to 6 to computational thinking and programming logic by allowing them to program a small wooden robot using colourful blocks. Children play with the “queue” of instructions, one of the fundamental concepts behind programming, without using literacy or screens.

The Cubetto Playset

The Robot: Cubetto is a cube-shaped wooden robot with a friendly engraved smile on its front. It moves on two wheels with two fixed plastic stands (called "castors") for stability.

Key features:

- Moves forward exactly 15 cm per instruction
- Rotates exactly 90° when turning
- Plays unique tunes for each command type
- Plays a longer tune when a sequence is complete

The Interface Board : The Interface Board is where children create their programmes by inserting instruction blocks into slots. You can distinguish:

- **Main Sequence (slots 1–12)**: Connected by an engraved line, these slots hold the main programme
- **Function Line (slots F1–F4)**: Enclosed in a separate area, used to create reusable sub-programmes

Appendixes - Tutorials of Robots Used in the Activity

Cubetto

Source: <https://primotoys.com/cubetto-user-manual/>

The Instruction Blocks: Four types of colourful wooden blocks, each representing a specific command:

Block	Colour	Shape	Action
Forward	Green	Arrow pointing up	Cubetto moves forward 15 cm
Left	Yellow	Arrow pointing left	Cubetto rotates 90° anticlockwise
Right	Red	Arrow pointing right	Cubetto rotates 90° clockwise
Function	Blue	Wavy line	Cubetto executes the function line

All blocks have a D-shaped base that fits into the board's slots, ensuring they can only be inserted one way.

The World Map: A 1 m × 1 m play mat featuring "Cubetto's World" – a 6×6 grid of 15 cm × 15 cm squares. Each square corresponds exactly to one forward movement by Cubetto.

Getting Started

Step 1: Insert Batteries

Both Cubetto and the Interface Board require 3 AA batteries each.

1. Locate the battery compartments on the bottom of each device
2. Use a crosshead screwdriver to open the compartments
3. Insert 3 AA batteries in each device, following the polarity markings
4. Replace the covers

Step 2: Power On

1. Find the sliding power switch on the bottom of each device
2. Slide to the ON position
3. Cubetto: Plays a welcome tune when powered on
4. Interface Board: All feedback LEDs blink three times when powered on

Appendixes - Tutorials of Robots Used in the Activity

Cubetto

Source: <https://primotoys.com/cubetto-user-manual/>

Step 3: Pair the Devices

Cubetto and the Interface Board communicate wirelessly and must be paired before use.

Pairing procedure:

1. Switch on the Interface Board first, then Cubetto
2. Insert any direction block (green, yellow, or red – not blue) into the first slot
3. Press the action button
4. Cubetto should perform the instruction
5. The devices are now paired!

Note: Pairing happens automatically in the background. Indoor wireless range is approximately 10 metres. Multiple Cubetto sets can be used in close proximity without interference.

If pairing fails: Turn both devices off, wait a few seconds, then repeat the procedure.

Programming Cubetto

1. Plan your route: Look at the map and decide where you want Cubetto to go
2. Choose your blocks: Select the instruction blocks you need
3. Build your sequence: Insert blocks into the Interface Board, starting from slot 1
4. Check the LEDs: Each correctly inserted block lights up its feedback LED
5. Press GO!: Press the action button to send the programme to Cubetto
6. Watch and learn: Cubetto executes each instruction in order

Important Rules

- Sequences start from slot 1 and follow the engraved line on the board
- No gaps allowed: Blocks inserted after an empty slot will not be recognised
- One programme at a time: Changing blocks during execution has no effect – wait until Cubetto finishes
- Feedback LEDs blink to show which instruction is currently being executed
- Unique tunes: Cubetto plays a different sound for each command type, helping children follow along

Appendixes - Tutorials of Robots Used in the Activity

Cubetto

Source: <https://primotoys.com/cubetto-user-manual/>

Using the Function Block

The blue Function Block is a powerful tool that allows children to pack multiple instructions into a single block. This introduces the concept of functions (or sub-programmes) – a fundamental idea in programming.

How Functions Work

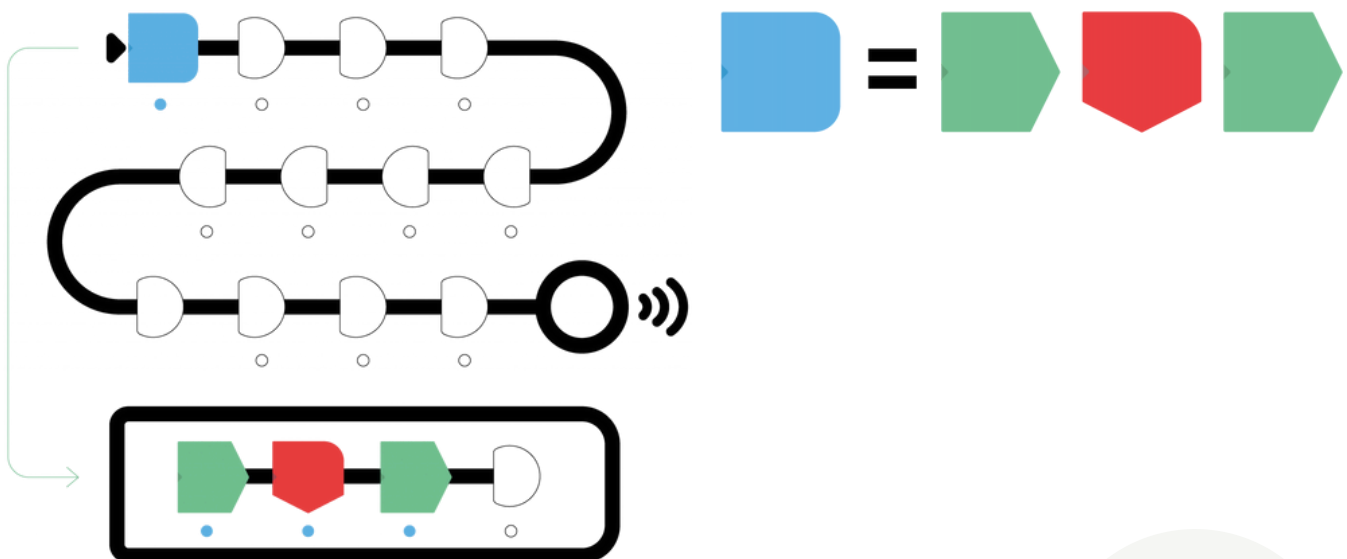
1. Create a function: Insert a sequence of blocks into the Function Line (the four slots at the bottom of the board)
2. Call the function: Place the blue Function Block anywhere in the main sequence
3. Execution: When Cubetto encounters the blue block, it executes all the instructions in the Function Line, then continues with the main sequence

Why Functions Matter

Functions teach children to:

- Identify patterns: "I keep doing the same thing over and over"
- Optimise: "How can I do this with fewer blocks?"
- Think abstractly: "This blue block means 'do the corner thing'"
- Solve complex problems: With only 12 main slots, functions allow longer programmes

Example: Drawing a Square with Functions



Appendixes - Tutorials of Robots Used in the Activity

Sphero Indi

Source: https://cdn.shopify.com/s/files/1/0306/6419/6141/files/Indi-Booklet_150x150_v15_outlined.pdf?v=1631191173



Sphero Indi is a programmable robot designed for young learners (ages 4+) that uses colour detection instead of traditional coding. Children programme Indi by placing coloured silicone tiles on the floor – the robot drives over them and responds to each colour with a specific behaviour. This screen-free, intuitive approach makes programming accessible to pre-readers whilst teaching computational thinking concepts.

Indi offers a unique **"IF colour THEN action"** programming logic: IF the robot sees green, THEN it goes forward; IF it sees purple, THEN it turns left. This conditional thinking is fundamental to all programming.

The Robot

Indi is a compact, car-shaped robot with a built-in colour sensor on its underside. The sensor reads the colour of tiles placed beneath the robot and triggers the corresponding behaviour.

Key features:

- Built-in colour sensor for detecting tiles
- LED lights that change to match detected colours
- Rechargeable battery (USB-C charging)
- Durable design for young learners
- Automatic rotation calibration

Appendixes - Tutorials of Robots Used in the Activity

Sphero Indi

Source: https://cdn.shopify.com/s/files/1/0306/6419/6141/files/Indi-Booklet_150x150_v15_outlined.pdf?v=1631191173

The Colour Tiles

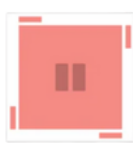
Indi comes with a set of colourful silicone tiles. Each colour triggers a specific default behaviour:



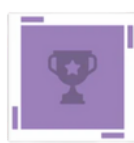
Go, go faster!



Slow down



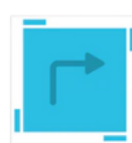
Stop



Celebrate!



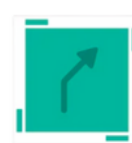
Turn 90° left



Turn 90° right



Turn 45° left



Turn 45° right

Getting Started

Step 1

Discover how indi reacts to each color tile.

Step 2

Build mazes with endless combinations of color tiles.

Step 3

Strengthen problem-solving skills by navigating the challenging puzzles you build.

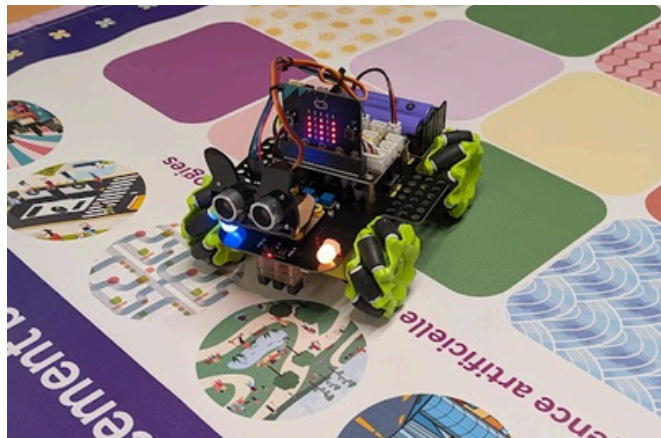
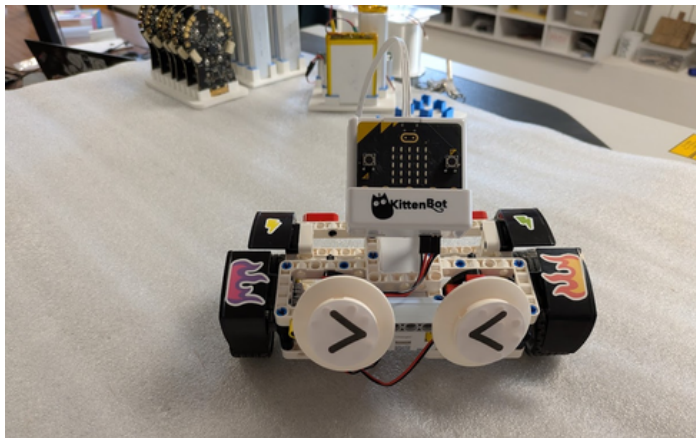


Make Your Own Colors

Some may wish to create their own color tiles, cards, or paths for indi. Keep in mind that each color is programmed into the firmware and instantly recognizable by indi. Other color sources, such as construction paper and markers, may not provide as consistent of an experience as using the color tiles and/or cards included with your indi(s). **The official Indi website provides all the pantone colours openly to use:** <https://sphero.com/pages/sphero-indi>. Feel free to create as many paths as possible and have fun!

Appendixes - Tutorials of Robots Used in the Activity

Coding a Micro:bit robot



Two examples of Micro:bit based robots used during the piloting phase of Robots Meet Arts. On the left, a robot created thanks to the [KittenBot PowerBrick 10-in-1 Robotics Kit for micro:bit](#) and on the right, a micro:bit robot based on the [KEYESTUDIO BBC Micro:bit 4WD Mecanum Robot Car Kit](#).

The **BBC micro:bit** is a versatile programming board that can be transformed into the brain of countless robot projects. Unlike Cubetto or Sphero Indi, which are self-contained systems, micro:bit robots are built from separate components: the micro:bit board itself, an expansion board (also called a "shield" or "breakout board"), motors, sensors, and a chassis or building system.

This modularity brings tremendous flexibility but also introduces an important concept: each robot platform requires to be programmed on MakeCode using specific extensions.

What is MakeCode?

MakeCode is a free, browser-based programming environment developed by Microsoft. It provides a visual, block-based approach to coding that makes programming accessible to beginners while offering text-based alternatives for more advanced users.

Key characteristics:

- Free and online: No installation required; works in any modern web browser
- Offline capable: A downloadable app is available for Windows, macOS, and Linux
- Multi-language: Supports blocks, JavaScript, and Python
- Simulator included: Test programmes without physical hardware
- Extensive documentation: Tutorials, examples, and teacher resources built in

Access MakeCode at: <https://makecode.microbit.org/>

Appendixes - Tutorials of Robots Used in the Activity

Coding a Micro:bit robot

Programming Modes

MakeCode offers three ways to write the same programme:

- **Blocks mode (default):** Visual programming using coloured, interlocking blocks. Ideal for beginners and younger pupils. Each block represents a programming concept without requiring memorisation of syntax.
- **JavaScript mode:** Text-based programming using JavaScript syntax. Blocks automatically convert to JavaScript, and vice versa. Useful for learners transitioning to text-based coding or for writing more complex logic.
- **Python mode:** Text-based programming using Python syntax. Available in newer versions of MakeCode. Particularly valuable as Python is widely taught in secondary education and used professionally.

The three modes are interconnected—changes in one mode reflect in the others (with some limitations for complex Python code). This allows gradual transition from visual to text-based programming.

For pupils aged 8 to 12, we recommend using the blocks mode as it will feel for children very logical and natural to give instructions to their robot.

If you are using a micro:bit for the first time

- Open MakeCode in your navigator.
- Connect the board to the computer: Use a micro-USB cable. The board will appear as a USB drive named "MICROBIT". Your board will be automatically associated with your editor on MakeCode. If not, you can manually associate it by clicking the "Connect device" button in the menu available by clicking "..." next to the "Download" button.
- Transfer the program: Once your code is created, click "Download" and the program will automatically download to the associated board. If not, you can copy the .hex file to the board. The orange LED blinks during transfer.
- Autonomous power: Add an AAA battery pack in the provided holder if not provided with your robotic kit.

For more information: [micro:bit - Getting Started - Introduction](#)

Appendixes - Tutorials of Robots Used in the Activity

Coding a Micro:bit robot

The Extension Ecosystem

The micro:bit's built-in MakeCode blocks cover basic functions: LED display, buttons, accelerometer, compass, radio, and simple pin input/output. **However, robot platforms use specialised hardware that requires custom programming blocks.**

Each manufacturer creates a MakeCode extension that provides:

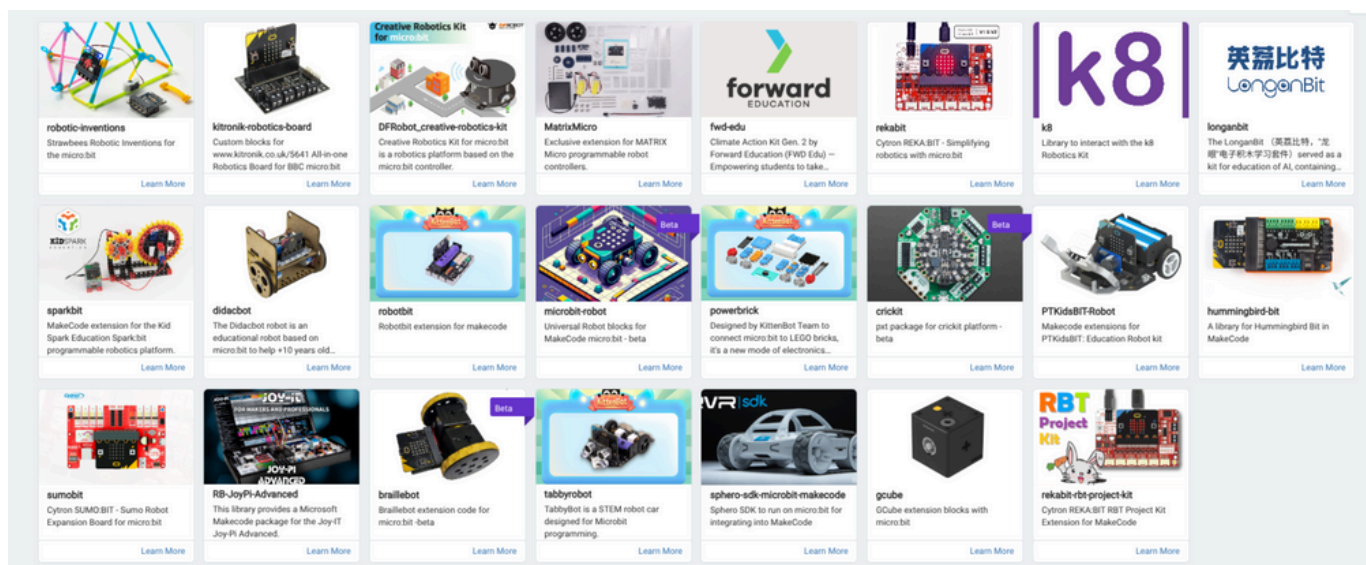
- Dedicated blocks for their specific motors and motor drivers
- Sensor blocks calibrated for their hardware
- Simplified commands that hide complex low-level code
- Visual blocks that match their product's terminology

Critical principle: A programme written with one robot's extension will not work with a different robot, even if both use micro:bit. The extension must match the hardware.

Adding Extensions in MakeCode

- Open MakeCode for micro:bit (<https://makecode.microbit.org/>)
- Create a new project or open an existing one
- Click Advanced in the block categories
- Click Extensions at the bottom
- Search for the extension name or paste the GitHub URL
- Click on the extension to add it to your project

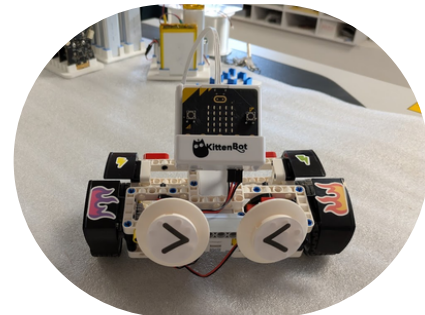
Once added, new block categories appear in the toolbox, typically in a distinctive colour.



Appendixes - Tutorials of Robots Used in the Activity

Example: KittenBot PowerBricks

As an example of what can be done with Micro:bit-based Robots, we are offering a specific tutorial related to the **KittenBot PowerBricks kit**, which we use in our workshops.



PowerBricks bridges the gap between the micro:bit and physical robotics through a simple principle: plug sensors and motors into numbered ports, then control them with dedicated MakeCode blocks. The system uses LEGO-compatible mounting, allowing pupils to design and build custom robot chassis using familiar construction techniques.

For robot projects, PowerBricks provides:

- **Movement:** DC motors for wheels and continuous rotation; servos for precise angular positioning (arms, grippers, steering)
- **Sensing distance:** Ultrasonic module measures obstacles from 2 to 400 cm
- **Detecting sound:** The same ultrasonic module includes a microphone for sound-reactive projects
- **Following lines:** Dual infrared sensors detect black lines on light surfaces
- **Detecting collisions:** Bumper switches trigger when pressed
- **Reading environment:** Temperature and humidity sensing for data-logging robots
- **Recognising colours:** Colour sensor returns hue values; gesture sensor detects hand movements
- **Playing audio:** MP3 module for voice feedback or sound effects

Adding the PowerBrick Extension

1. In MakeCode, click Advanced → Extensions
2. Search for powerbrick
3. Select KittenBot PowerBrick from the results
4. The Powerbrick category appears in orange

Building a Robot with PowerBricks

A typical PowerBricks robot is built based on the Lego technical bricks and uses two DC motors (M1 and M2) driving left and right wheels for differential steering. You can add one ultrasonic sensor at the front for obstacle detection but it is not needed for this project.

Appendixes - Tutorials of Robots Used in the Activity

Example: KittenBot PowerBricks

Movement Principles

Differential steering controls the robot by varying the speed of each wheel:

Left Motor (M1)	Right Motor (M2)	Result
Forward	Forward	Straight ahead
Backward	Backward	Straight reverse
Stop	Forward	Pivot left
Forward	Stop	Pivot right
Backward	Forward	Spin left on the spot
Forward	Backward	Spin right on the spot

Speed values range from -255 (full reverse) to 255 (full forward). Zero stops the motor.

Appendixes - Tutorials of Robots Used in the Activity

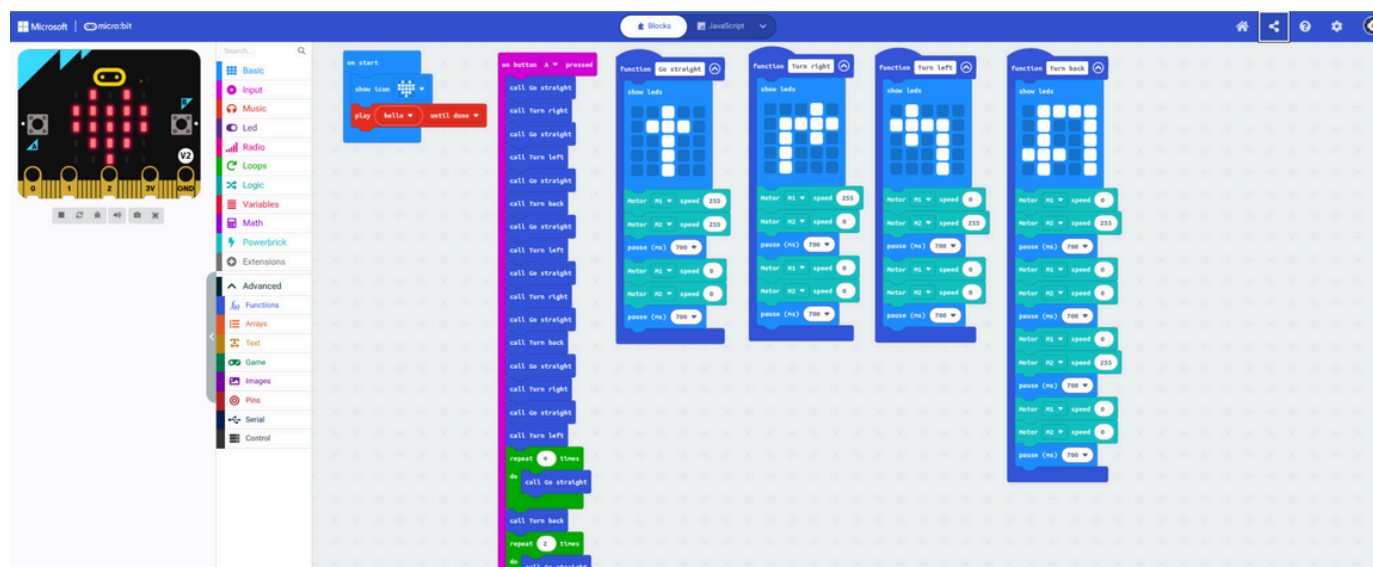
Example: KittenBot PowerBricks

Grid Navigation Programme: A Classroom Example

The following programme demonstrates a robot navigating a predefined path on a grid. It was developed for use with the Bio-Inspired Learning curriculum, where pupils first memorise a route themselves, then programme the robot to follow the same path.

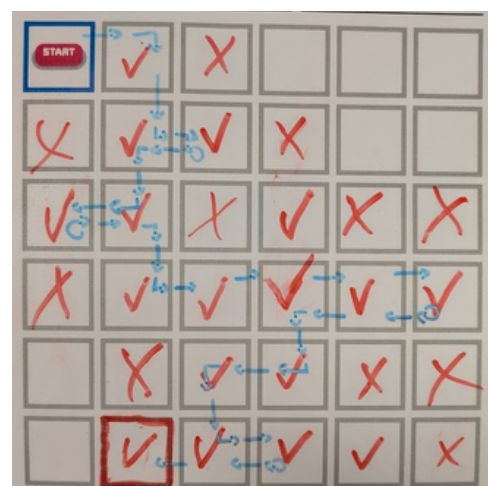
This real classroom example illustrates several key programming concepts: functions, sequential execution, loops, visual feedback, and timing calibration.

The programme is visible here: <https://makecode.microbit.org/S25194-48798-31124-89886>



In this sequence, the robot need to go from “Start” to the arrival visible on the example memorisation sheet in case B6.

In red, you can see the information of the model created by the pupil, and in blue, the arrows describe the path the student wants the robot to take with this objective: ***from the start, reach the arrival by crossing as many cases as possible.***



Appendixes - Tutorials of Robots Used in the Activity

Example: KittenBot PowerBricks

Part 1: Programme Structure

The programme is organised into distinct sections:

1. **Startup code:** Runs once when the micro:bit powers on
2. **Function definitions** (Turn_left, Turn_right, Go_straight, Turn_back): Reusable movement primitives
3. **Event handler** (input.onButtonPressed): The main navigation sequence triggered by button A

This structure follows good programming practice: **define your tools first, then use them in the main programme.**

Organising Code with Functions

Rather than scattering motor commands throughout the programme, we define **four movement functions**: Go_straight, Turn_left, Turn_right, and Turn_back. Each function encapsulates a **complete action**—displaying a directional arrow on the LEDs, running the appropriate motors for a calibrated duration, then stopping and pausing before the next movement.

This approach transforms the main programme into something **readable**. Instead of deciphering powerbrick.MotorRun(M1, 255) followed by powerbrick.MotorRun(M2, 255), we simply read Go_straight(). The code becomes a **narrative of the robot's journey** that pupils can follow without understanding the underlying motor control.

Functions also **simplify maintenance**. If the robot consistently overshoots grid cells, we adjust the timing in one place rather than hunting through dozens of scattered pause values. When debugging, we can test each function in isolation before combining them into complex sequences.

Loops for Cleaner Code

When the robot must travel four cells in a row, we could write Go_straight four times. Instead, a **loop repeats the function call**, making the intent crystal clear: "move forward four times." Beyond tidiness, loops reduce errors—there is no risk of accidentally copying three times instead of four—and they introduce the fundamental concept of **iteration** that pupils will encounter throughout their programming journey.

Appendixes - Tutorials of Robots Used in the Activity

Example: KittenBot PowerBricks

Visual Feedback and Calibration

Each movement displays a **custom LED arrow matching the direction**, allowing teachers and pupils to "read" the robot's intentions. If the display shows a left arrow but the robot turns right, the diagnosis is immediate: the motor connections are swapped.

Programme Flow

On startup, a **heart icon and greeting sound confirm the robot is ready and waiting**. **Pressing button A launches the navigation sequence**. Upon completion, a chessboard pattern and celebratory sound provide closure. This clear structure—ready, running, finished—helps pupils understand programme states and gives satisfying feedback for their work.

